JonesAuto Database Management System

CPSC 3660

Gurmann Basran

April 8, 2026

1 Stage 1: Overall Design

JonesAuto is a used car dealership in Lethbridge that buys cars at auctions, fixes them up, and resells them to customers. A lot of their sales are financed so they have to track monthly payments on top of everything else. I built a database system to replace their paper-based workflow. It covers the full lifecycle of a car: buying it at auction, recording repairs, selling it to a customer, setting up warranty coverage, and tracking payments. The system runs on PHP and MySQL on Linux using PHP's built-in development server. There are 6 input forms, 6 output reports, and 12 queries total, with 7 of those queries using advanced SQL features like JOINs, subqueries, and GROUP BY with HAVING.

1.1 System Goals

The main goal was to replace JonesAuto's paper records with a proper database system. The system needs to track the full car lifecycle from purchase at auction all the way through to customer payments. It also needs to give employees quick access to customer info, vehicle history, and financial summaries without digging through filing cabinets.

1.2 Input Forms

Form	**Purpose**
Purchase Form	Records a car bought at auction with vehicle details, price, buyer, repairs
Sale Form	Records a car sale with financing, customer info, employment history
Warranty Form	Adds warranty coverage to an existing sale
Payment Form	Records monthly payments, detects late payments, updates credit stats
Customer Form	Adds/edits customers with soft-delete support
Employee Form	Adds/edits employees (buyers/salespeople) with soft-delete support

1.3 Output Reports

Report	Purpose
Current Inventory	Shows available cars on the lot, filterable by make, with repair cost totals
Sales Summary	Shows completed sales with profit calculation, filterable by date range
Payment History	Shows payment records for a selected customer, highlights late payments
Repair Costs	Compares estimated vs actual repair costs, highlights overruns
Active Warranties	Lists warranty coverage with color-coded expiry urgency
Late Payments	Shows customers with late payments, sorted worst-first

1.4 System Users

There are three types of users: buyer employees who go to auctions and fill out purchase forms, salesperson employees who handle sales and warranties and payments, and office staff who run reports and queries. The system doesn't have a login since it is a small dealership and everyone pretty much shares the same machine.

1.5 Initial Design Assumptions

- Cars are only acquired through auctions (though there's an `is_auction` flag for flexibility)
- Each car has exactly one purchase record and at most one sale
- Customers can buy multiple cars over time
- Warranties are tied to a specific sale, not directly to a customer
- The dealership is in Lethbridge, Alberta (all sample data uses Alberta locations)
- Payments are monthly, and "late" means `paid_date` is after `due_date`
- All tables use an `is_active` flag for soft-delete instead of actually removing rows

I made these assumptions based on the project description. Some of them I had to figure out myself since not everything was spelled out.

2 Stage 2: ER Diagram and Database Schema

We covered ER diagrams in class using Chen notation, where rectangles represent entities, diamonds represent relationships, and ovals represent attributes. I designed the ER diagram before writing any SQL since I could map the ER model directly to relational tables, giving a more structured approach than jumping straight to CREATE TABLE statements.

The full ER diagram for JonesAuto is on the following page (Figure 1). The diagram shows all 10 entity sets, the 11 relationships between them, primary keys, and cardinality labels. Double lines mean the entity must participate in the relationship, single lines mean it's optional.

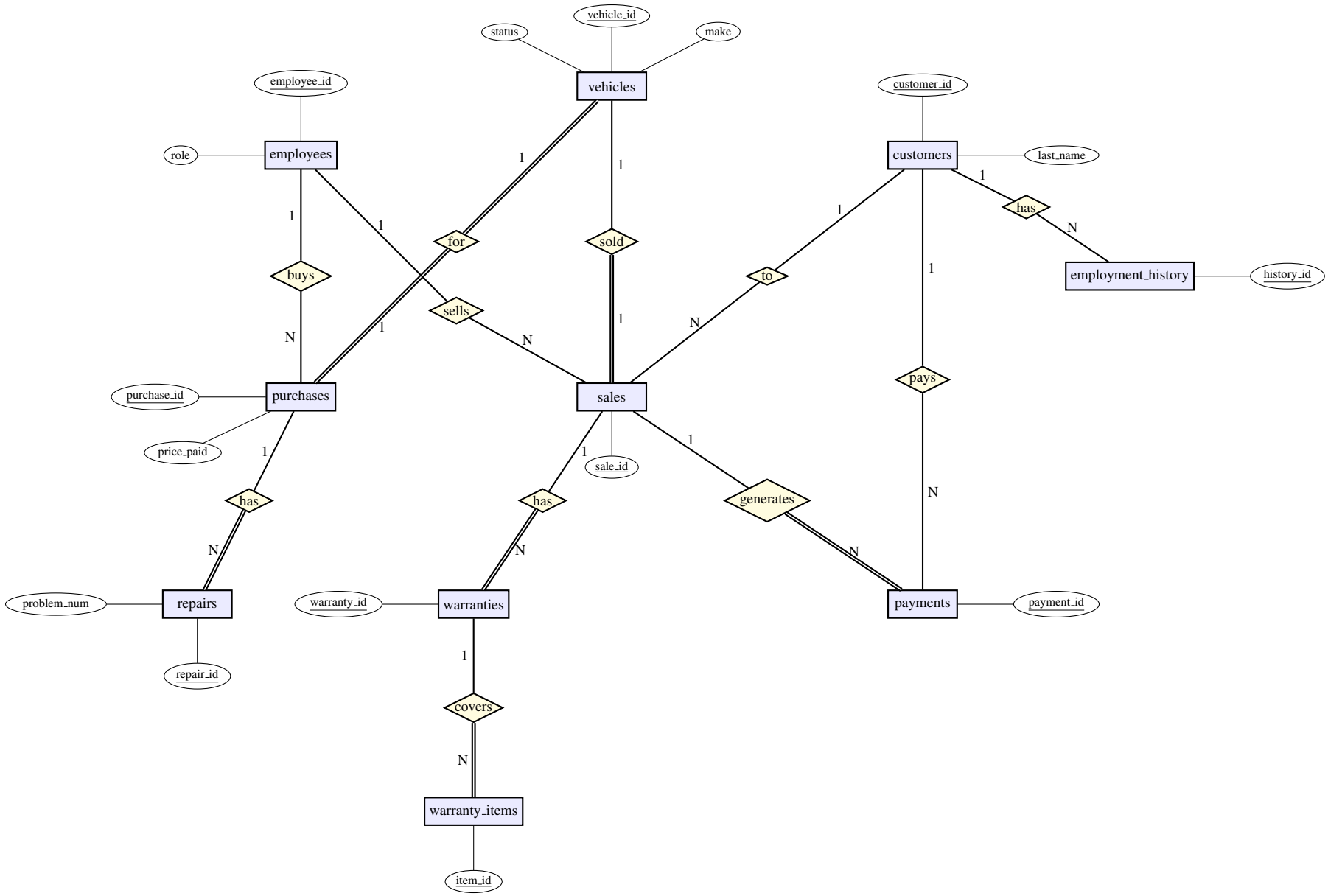


Figure 1: Entity-Relationship Diagram for JonesAuto Database (Chen Notation)

2.1 Entities and Attributes

Entity	Attributes	Primary Key
employees	employee_id, first_name, last_name, phone, role, is_active	employee_id
vehicles	vehicle_id, make, model, year, color, miles, condition_desc, book_price, style, interior_color, status, is_active	vehicle_id
purchases	purchase_id, vehicle_id, employee_id, purchase_date, location, seller_dealer, is_auction, price_paid, is_active	purchase_id
repairs	repair_id, purchase_id, problem_num, description, est_cost, actual_cost, is_active	repair_id
customers	customer_id, first_name, last_name, phone, address, city, state, zip, gender, dob, num_late_payments, avg_days_late, is_active	customer_id
employment_history	history_id, customer_id, employer, title, supervisor_phone, employer_address, start_date, is_active	history_id
sales	sale_id, vehicle_id, customer_id, employee_id, sale_date, total_due, down_payment, financed_amount, sale_price, commission, is_active	sale_id
warranties	warranty_id, sale_id, vehicle_id, customer_id, employee_id, warranty_sale_date, total_cost, monthly_cost, is_active	warranty_id
warranty_items	item_id, warranty_id, warranty_type, start_date, length_months, cost, deductible, items_covered, is_active	item_id
payments	payment_id, customer_id, sale_id, payment_date, due_date, paid_date, amount, bank_account, is_active	payment_id
operations_log	log_id, table_name, record_id, operation, old_values, new_values, changed_at	log_id

2.2 Relationships

1. **employees to purchases** : one buyer employee can make many purchases
2. **vehicles to purchases** : each vehicle has exactly one purchase record
3. **purchases to repairs** : one purchase can have multiple repair problems
4. **customers to employment_history** : each customer can have multiple employer records
5. **vehicles to sales** : each vehicle can be sold once
6. **customers to sales** : one customer can buy multiple cars
7. **employees to sales** : one salesperson can handle many sales
8. **sales to warranties** : one sale can have multiple warranty contracts
9. **warranties to warranty_items** : each warranty can cover multiple items
10. **sales to payments** : one sale generates multiple monthly payments
11. **customers to payments** : denormalized link for quick customer payment lookup

Total: 14 foreign key constraints across the schema.

2.3 Relational Schema

Figure 2 shows the relational schema after mapping the ER model to tables. Each box represents a table, and the arrows show foreign key references from child to parent.

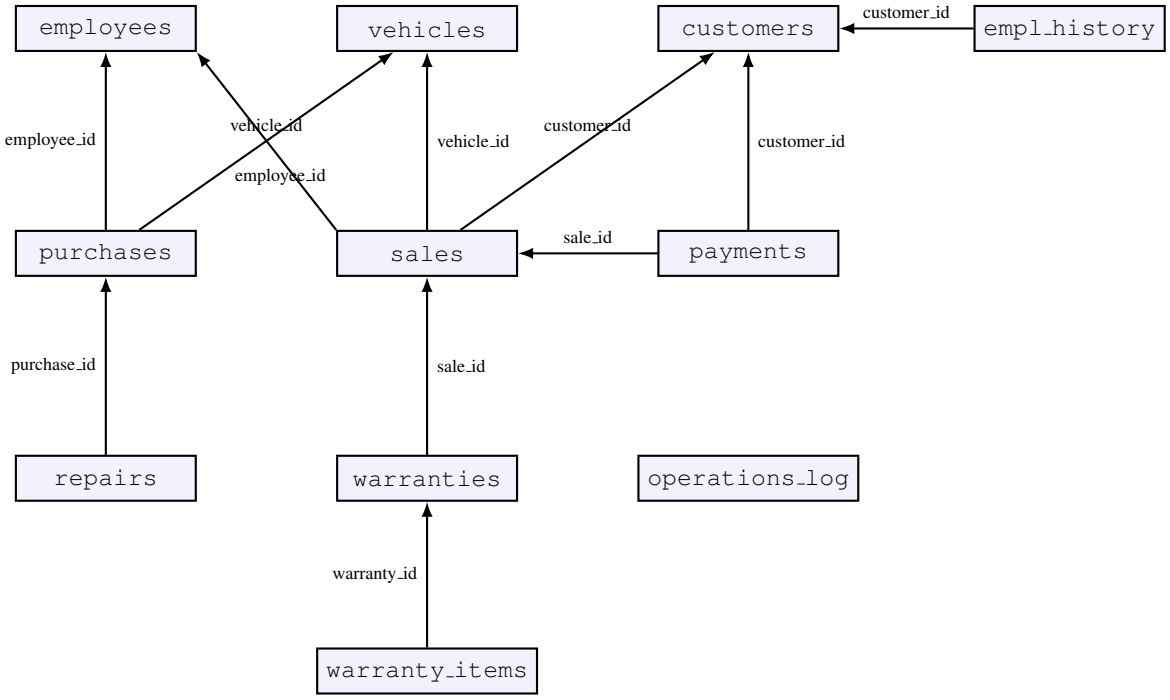


Figure 2: Relational Schema with Foreign Key Relationships

2.4 Normalization

We went through normalization in class covering 1NF, 2NF, 3NF, and BCNF. All of my tables are in 3NF. There are no partial dependencies since every primary key is a single-column AUTO_INCREMENT integer, so 2NF is satisfied automatically. There are no transitive dependencies either since each non-key attribute depends directly on the primary key.

That said, I did make two deliberate denormalizations. The first is `num_late_payments` and `avg_days_late` on the customers table. Technically I could calculate these from the payments table every time, but the payment form spec required showing them. Recalculating with an aggregate query on every page load seemed slow and unnecessary, so I stored them directly on the customer record. It's just a trade-off between normalization and performance.

The second is `customer_id` on the payments table. That info already exists on the sales table (which payments link to through `sale_id`), so it's redundant. But having it directly on payments means I don't need to JOIN through sales every time I want to look up a customer's payment history. Just makes queries simpler.

BCNF is stricter than 3NF, but for this project 3NF was the right target. The two denormalizations are documented and intentional.

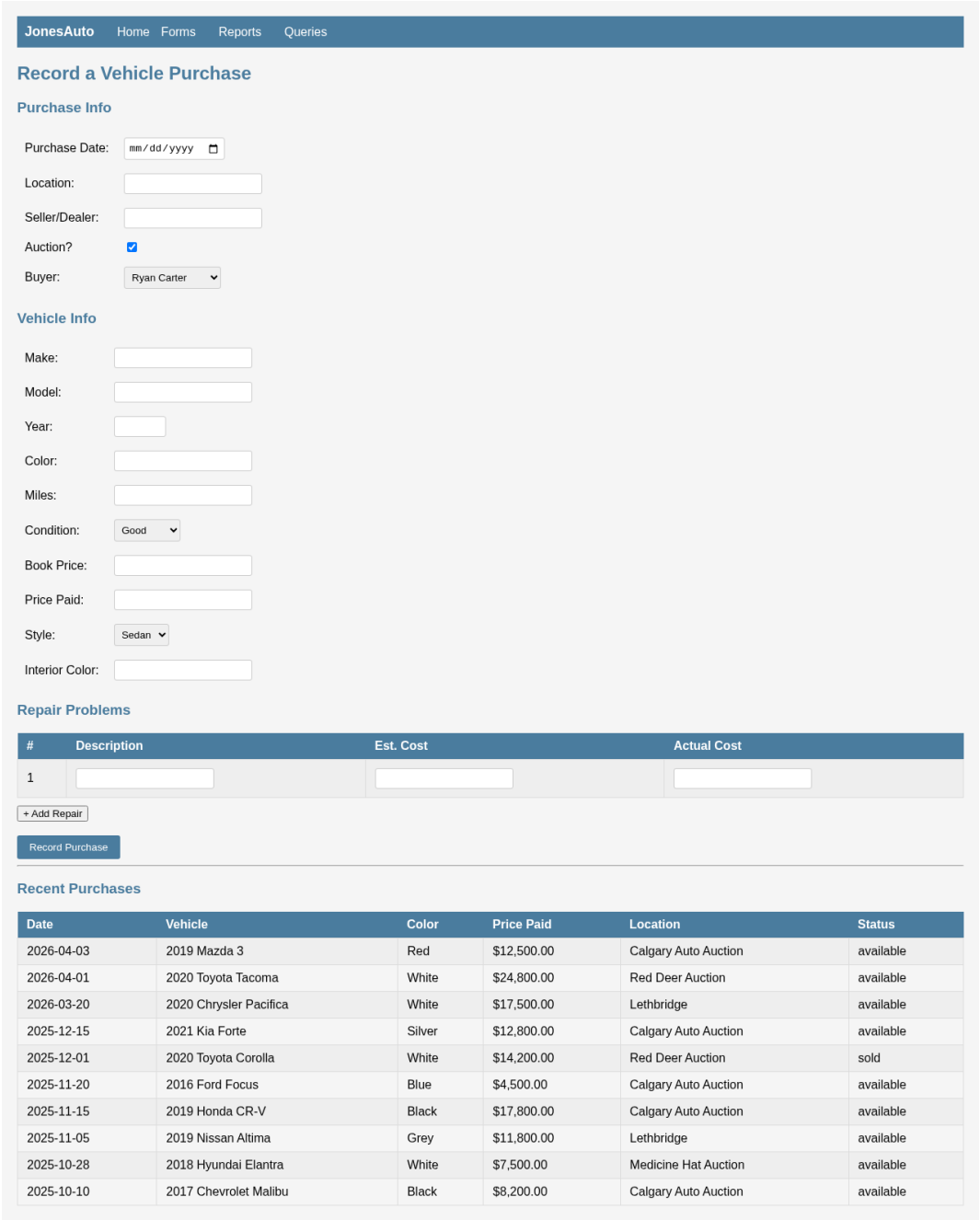
3 Stage 3: Forms and Reports

This is where most of the coding happened. I built 6 forms and 6 reports in PHP with mysqli. Each form has two files: one for the HTML and one for processing the submitted data.

3.1 Input Forms

Purchase Form (`purchase_form.php`)

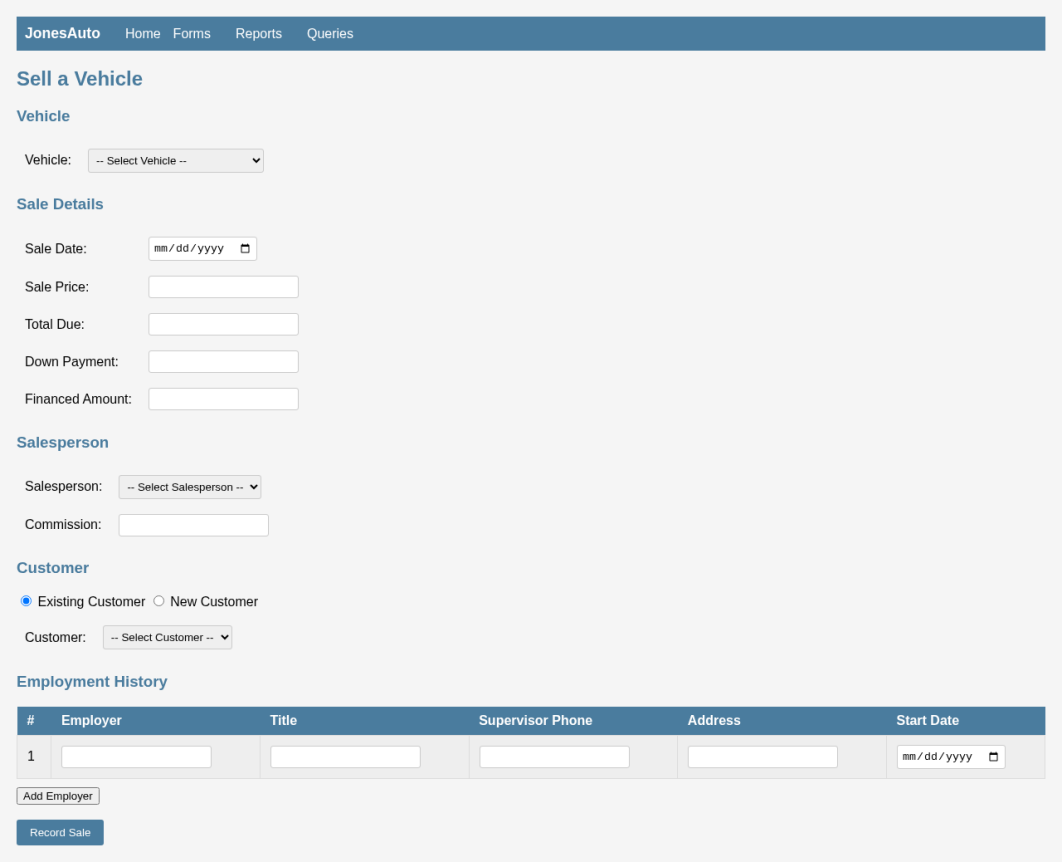
Records a car bought at auction. You pick a buyer from the dropdown, fill in the purchase and vehicle details, and add repairs at the bottom. Instead of a fixed number of repair rows, there's a dynamic "Add Repair" button that lets you add as many repair problems as needed. It saves to the vehicles, purchases, and repairs tables.



Sale Form (`sale_form.php`)

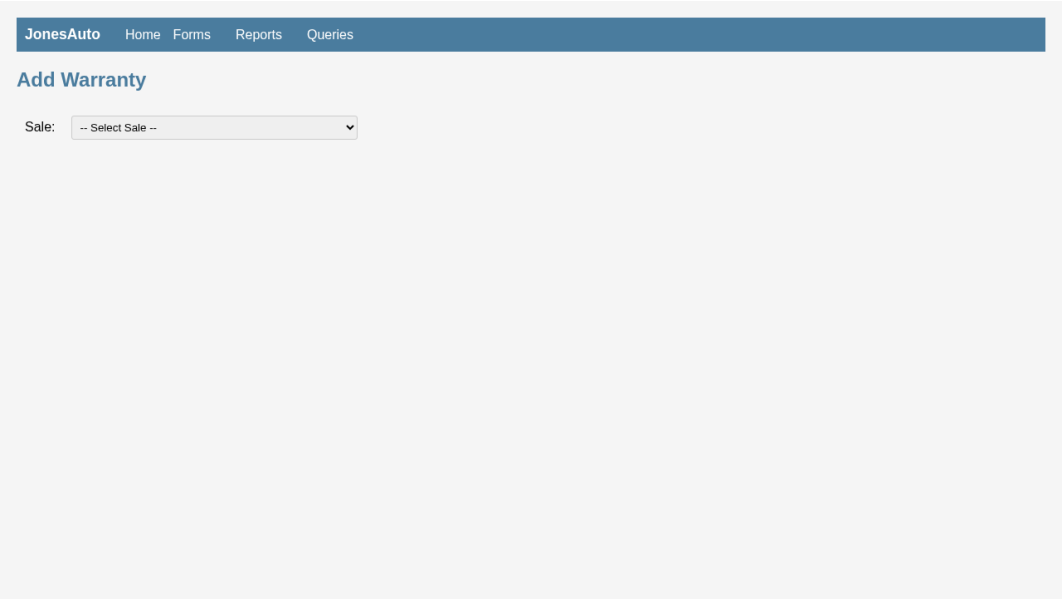
This one records a car sale. There's a toggle at the top for choosing between an existing customer and a new one. If you pick "New Customer," a set of fields appears for entering their info: name, phone, address, city, state, zip, gender, and date of birth. If you pick "Existing Customer," you just select from a dropdown. The vehicle is selected from a list of available cars, and when you pick one it shows the book price, miles, and condition so the salesperson has that info handy. If you pick an existing customer, their credit info (late payments, average days late) shows up too. Then there are the sale details: date, sale price, total due, down payment, financed amount, and commission.

At the bottom there's space for employment history entries for the customer. The tricky part was the existing/new customer toggle since it changes which fields show up on the page. Submitting inserts into sales and employment_history, creates a new customer record if needed, and updates the vehicle status to sold.



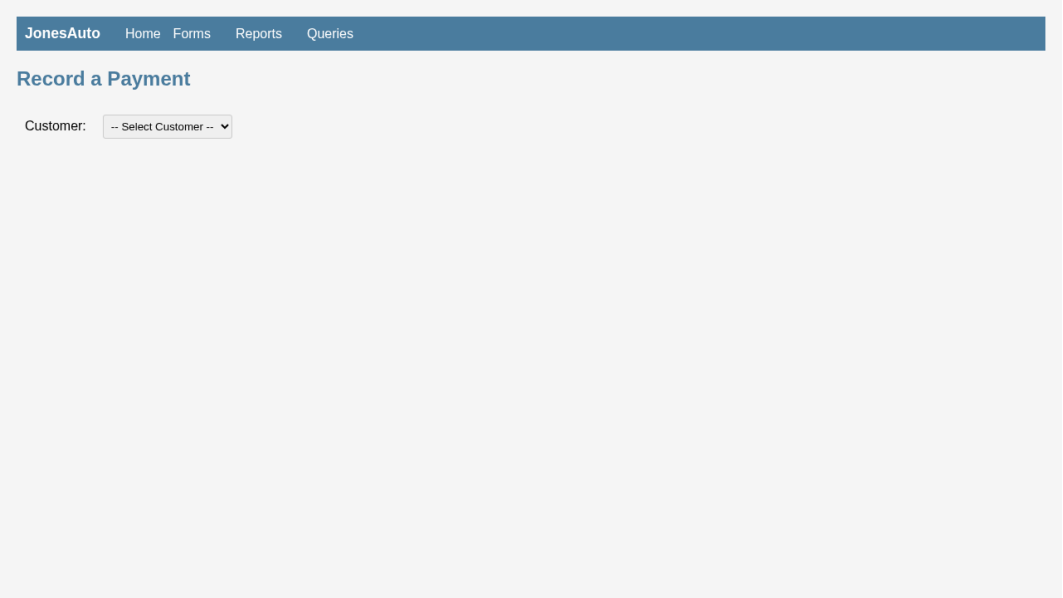
Warranty Form (`warranty_form.php`)

Adds warranty coverage to a sale. Pick a sale from the dropdown and the vehicle/customer info fills in automatically. If the sale already has warranties, those show up on the page so you can see what coverage already exists before adding more. Then enter the warranty details and add coverage items with type, dates, and costs.



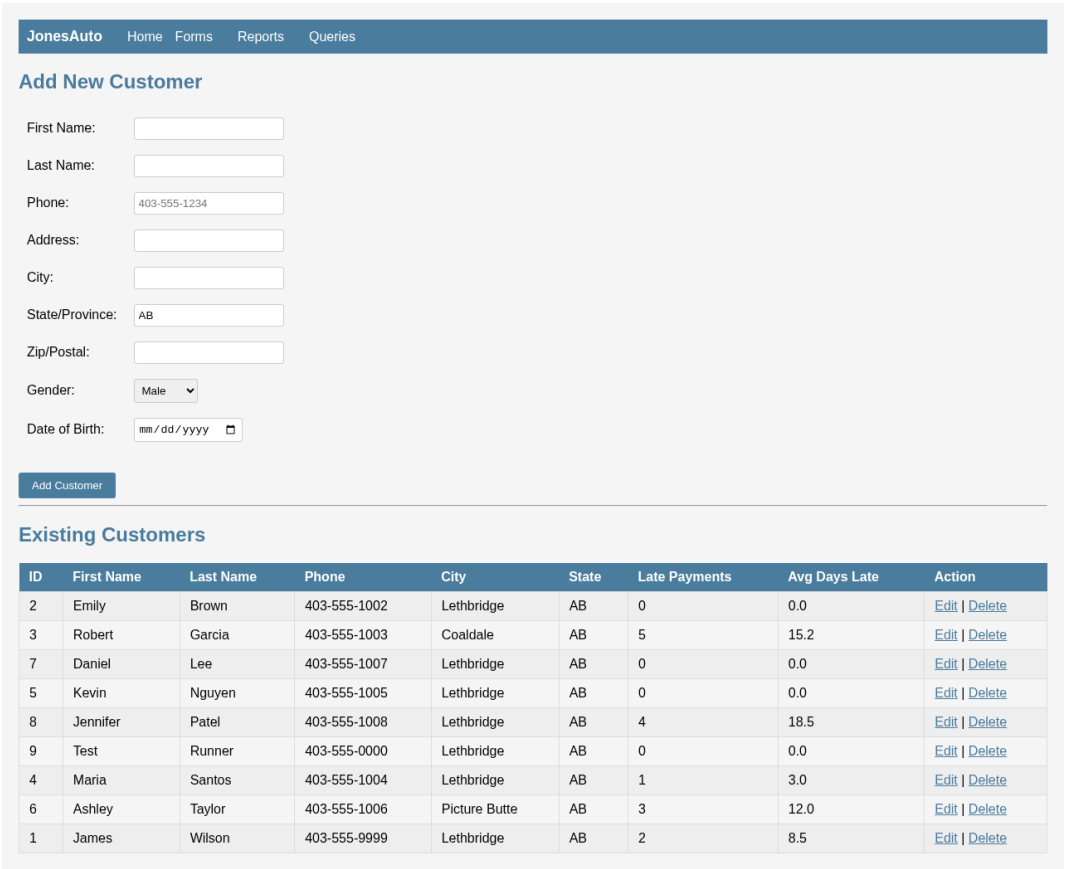
Payment Form (`payment_form.php`)

This form records a customer payment. There's a customer dropdown at the top that reloads the page when you select someone, so it can show that customer's info and existing payments below. Once a customer is selected, you see their gender, date of birth, number of late payments, and average days late. Then you pick which sale the payment is for, and enter the payment date, due date, paid date, amount, and bank account. If the paid date is after the due date, it gets flagged as late and the customer's late payment stats get recalculated. Below the form there is a table showing all existing payments for that customer, with late ones highlighted in red.



Customer Form (`customer_form.php`)

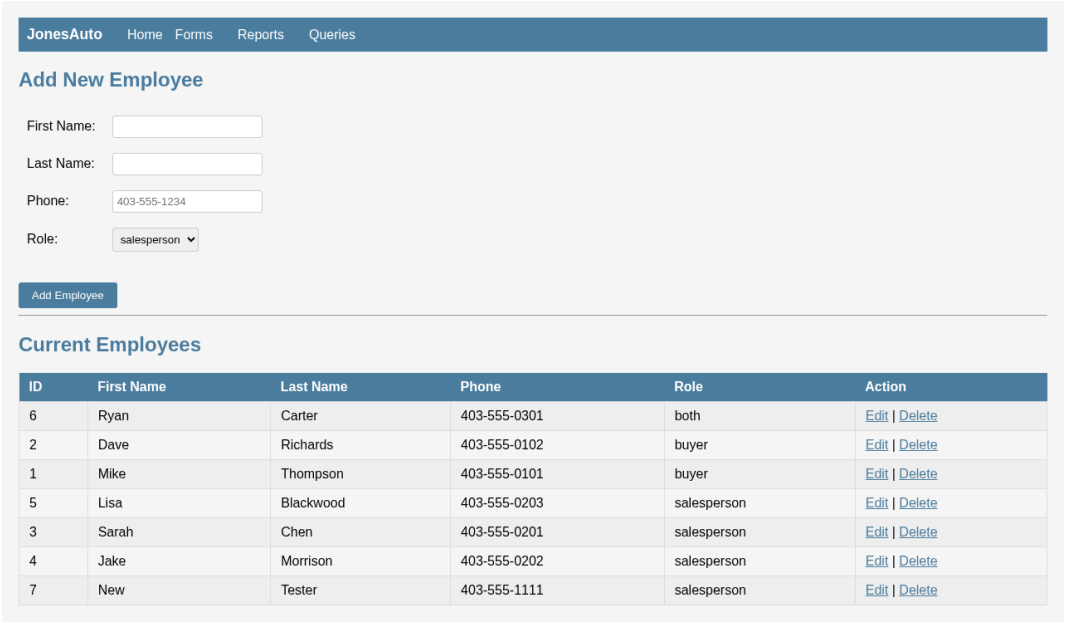
This form handles adding, editing, and soft-deleting customers. The top half has input fields for first name, last name, phone, address, city, state/province, zip/postal code, gender, and date of birth. You can also click on an existing customer to load their info into the form for editing. There's a deactivate button that sets the `is_active` flag to 0 instead of actually deleting the row. Below the form there is a table showing all active customers with their contact info, late payment count, and average days late.



ID	First Name	Last Name	Phone	City	State	Late Payments	Avg Days Late	Action
2	Emily	Brown	403-555-1002	Lethbridge	AB	0	0.0	Edit | Delete
3	Robert	Garcia	403-555-1003	Coaldale	AB	5	15.2	Edit | Delete
7	Daniel	Lee	403-555-1007	Lethbridge	AB	0	0.0	Edit | Delete
5	Kevin	Nguyen	403-555-1005	Lethbridge	AB	0	0.0	Edit | Delete
8	Jennifer	Patel	403-555-1008	Lethbridge	AB	4	18.5	Edit | Delete
9	Test	Runner	403-555-0000	Lethbridge	AB	0	0.0	Edit | Delete
4	Maria	Santos	403-555-1004	Lethbridge	AB	1	3.0	Edit | Delete
6	Ashley	Taylor	403-555-1006	Picture Butte	AB	3	12.0	Edit | Delete
1	James	Wilson	403-555-9999	Lethbridge	AB	2	8.5	Edit | Delete

Employee Form (`employee_form.php`)

Same idea as the customer form but for employees. Enter name, phone, and role. You can edit existing employees and deactivate them with soft-delete. Existing active employees are listed below.



3.2 Output Reports

The reports pull data from the database and display it in tables. Most of them have filters at the top, and several include action links so you can jump straight to related forms.

Current Inventory (`inventory.php`)

Shows all vehicles with status "available." There's a text input at the top for filtering by make. For each car it shows make, model, year, color, miles, condition, book price, price paid, and total repair costs summed from the repairs table. It also calculates a total cost column (price paid plus repairs). Each row has a "Sell" link that takes you to the sale form with that vehicle pre-selected. The query uses a JOIN between vehicles and purchases, plus a subquery on repairs.

JonesAuto Home Forms Reports Queries

Current Inventory

Filter by Make: Filter

Showing: All Makes

Make	Model	Year	Color	Miles	Condition	Book Price	Price Paid	Repair Costs	Total Cost	Action
Chevrolet	Malibu	2017	Black	110,000	Fair	$12,500.00	$8,200.00	$270.00	$8,470.00	Sell
Chrysler	Pacifica	2020	White	62,000	Good	$22,000.00	$17,500.00	$0.00	$17,500.00	Sell
Ford	Focus	2016	Blue	135,000	Fair	$7,200.00	$4,500.00	$380.00	$4,880.00	Sell
Honda	Civic	2020	Blue	45,000	Good	$17,200.00	$13,500.00	$0.00	$13,500.00	Sell
Honda	CR-V	2019	Black	71,000	Good	$22,500.00	$17,800.00	$170.00	$17,970.00	Sell
Hyundai	Elantra	2018	White	88,000	Fair	$11,800.00	$7,500.00	$625.00	$8,125.00	Sell
Kia	Forte	2021	Silver	29,000	Excellent	$16,200.00	$12,800.00	$0.00	$12,800.00	Sell
Mazda	3	2019	Red	58,000	Good	$16,500.00	$12,500.00	$310.00	$12,810.00	Sell
Nissan	Altima	2019	Grey	67,000	Good	$15,900.00	$11,800.00	$0.00	$11,800.00	Sell
Subaru	Outback	2018	Green	93,000	Good	$19,800.00	$15,500.00	$280.00	$15,780.00	Sell
Toyota	Camry	2019	Silver	78,000	Good	$18,500.00	$14,200.00	$470.00	$14,670.00	Sell
Toyota	Tacoma	2020	White	47,000	Good	$30,200.00	$24,800.00	$95.00	$24,895.00	Sell

12 vehicle(s) found.

Sales Summary (`sales_report.php`)

Shows all completed sales. You can filter by date range using a from/to date picker. For each sale it shows the date, vehicle info, customer name, salesperson, sale price, purchase cost, repair costs, and profit. The profit calculation was one of the trickier queries since it pulls from vehicles, purchases, repairs, sales, customers, and employees. Profit numbers are color-coded green for positive and red for negative. Each row has an "Add Warranty" link. There is a totals row at the bottom.

JonesAuto Home Forms Reports Queries

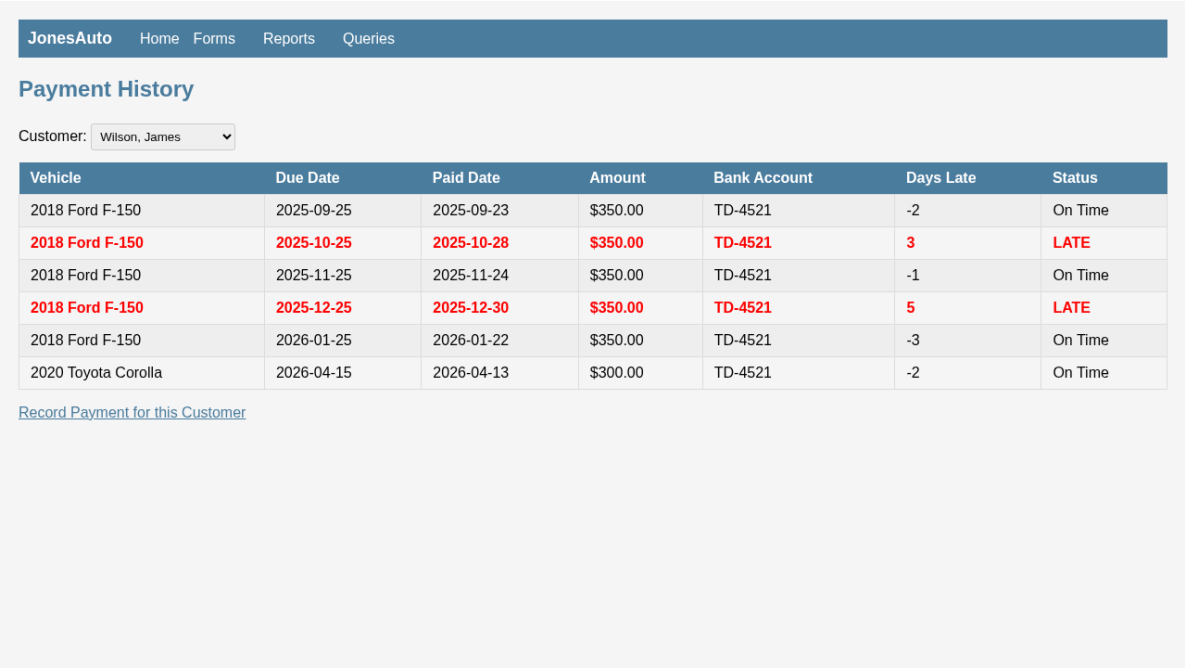
Sales Report

From: mm/dd/yyyy To: mm/dd/yyyy Filter

Date	Vehicle	Customer	Salesperson	Sale Price	Cost	Repairs	Profit	Action
2026-03-15	2020 Toyota Corolla	James Wilson	Jake Morrison	$19,500.00	$14,200.00	$0.00	**$5,300.00**	Add Warranty
2025-10-25	2019 Dodge Ram 1500	Kevin Nguyen	Jake Morrison	$32,500.00	$22,000.00	$280.00	**$10,220.00**	Add Warranty
2025-10-15	2021 Toyota RAV4	Emily Brown	Jake Morrison	$29,500.00	$21,500.00	$90.00	**$7,910.00**	Add Warranty
2025-09-30	2017 Honda Accord	Ashley Taylor	Sarah Chen	$16,800.00	$10,200.00	$190.00	**$6,410.00**	Add Warranty
2025-09-20	2020 Ford Escape	Robert Garcia	Sarah Chen	$22,800.00	$15,200.00	$470.00	**$7,130.00**	Add Warranty
2025-08-25	2018 Ford F-150	James Wilson	Sarah Chen	$26,500.00	$18,500.00	$710.00	**$7,290.00**	Add Warranty
2025-08-10	2017 Chevrolet Cruze	Maria Santos	Lisa Blackwood	$11,200.00	$5,200.00	$675.00	**$5,325.00**	Add Warranty
Totals				**$158,800.00**	**$106,800.00**	**$2,415.00**	**$49,585.00**	

Payment History (`payment_history.php`)

Pick a customer from the dropdown and it shows all their payments. Late ones are highlighted in red. There's a "Record Payment" link to jump to the payment form.



JonesAuto Home Forms Reports Queries

Payment History

Customer: Wilson, James

Vehicle	Due Date	Paid Date	Amount	Bank Account	Days Late	Status
2018 Ford F-150	2025-09-25	2025-09-23	$350.00	TD-4521	-2	On Time
2018 Ford F-150	**2025-10-25**	**2025-10-28**	**$350.00**	**TD-4521**	**3**	**LATE**
2018 Ford F-150	2025-11-25	2025-11-24	$350.00	TD-4521	-1	On Time
2018 Ford F-150	**2025-12-25**	**2025-12-30**	**$350.00**	**TD-4521**	**5**	**LATE**
2018 Ford F-150	2026-01-25	2026-01-22	$350.00	TD-4521	-3	On Time
2020 Toyota Corolla	2026-04-15	2026-04-13	$300.00	TD-4521	-2	On Time

Record Payment for this Customer

Repair Costs (`repair_summary.php`)

Shows estimated vs actual repair costs for every repair. Over-budget repairs are highlighted so the dealership can see which cars cost more to fix than expected.

JonesAuto Home Forms Reports Queries

Repair Cost Summary

Vehicle	Purchase Date	Problem #	Description	Est. Cost	Actual Cost	Difference
2019 Toyota Camry	2025-08-15	1	Scratch on rear bumper	$150.00	$120.00	$-30.00
2019 Toyota Camry	2025-08-15	2	Dent on driver door	$300.00	$350.00	+$50.00
2018 Ford F-150	2025-07-20	1	Cracked windshield	$400.00	$380.00	$-20.00
2018 Ford F-150	2025-07-20	2	Rust spot on wheel well	$200.00	$250.00	+$50.00
2018 Ford F-150	2025-07-20	3	Interior stain on back seat	$100.00	$80.00	$-20.00
2017 Chevrolet Malibu	2025-10-10	1	Paint chip on hood	$150.00	$160.00	+$10.00
2017 Chevrolet Malibu	2025-10-10	2	Broken side mirror	$120.00	$110.00	$-10.00
2021 Toyota RAV4	2025-09-25	1	Minor scratch on fender	$100.00	$90.00	$-10.00
2020 Ford Escape	2025-08-30	1	Dent on rear quarter panel	$250.00	$280.00	+$30.00
2020 Ford Escape	2025-08-30	2	Worn brake pads	$200.00	$190.00	$-10.00
2018 Hyundai Elantra	2025-10-28	1	Broken tail light	$80.00	$75.00	$-5.00
2018 Hyundai Elantra	2025-10-28	2	Seat tear on passenger side	$150.00	$200.00	+$50.00
2018 Hyundai Elantra	2025-10-28	3	AC not blowing cold	$300.00	$350.00	+$50.00
2019 Honda CR-V	2025-11-15	1	Scratch on passenger door	$180.00	$170.00	$-10.00
2017 Chevrolet Cruze	2025-07-05	1	Multiple body dents	$500.00	$620.00	+$120.00
2017 Chevrolet Cruze	2025-07-05	2	Headlight foggy	$60.00	$55.00	$-5.00
2016 Ford Focus	2025-11-20	1	Bumper crack	$200.00	$180.00	$-20.00
2016 Ford Focus	2025-11-20	2	Power window not working	$150.00	$200.00	+$50.00
2019 Dodge Ram 1500	2025-09-18	1	Rust on undercarriage	$300.00	$280.00	$-20.00
2018 Subaru Outback	2025-10-05	1	Paint fading on roof	$250.00	$240.00	$-10.00
2018 Subaru Outback	2025-10-05	2	Door handle loose	$50.00	$40.00	$-10.00
2017 Honda Accord	2025-08-12	1	Dent on trunk	$200.00	$190.00	$-10.00
2020 Toyota Tacoma	2026-04-01	1	Chip in windshield	$100.00	$95.00	$-5.00
2019 Mazda 3	2026-04-03	1	Scratch on hood	$120.00	$130.00	+$10.00
2019 Mazda 3	2026-04-03	2	Interior needs detailing	$200.00	$180.00	$-20.00

Active Warranties (`warranty_report.php`)

Lists all warranty items with their expiry info. The page color-codes rows by urgency: red background for expired warranties, yellow background for ones expiring within 30 days, and no color for ones with more time left. Each row shows customer name, vehicle, warranty type, start date, length in months, expiry date, days remaining, cost, deductible, and items covered. The query JOINs warranties, warranty_items, vehicles, and customers.

JonesAuto Home Forms Reports Queries

Active Warranties

Red = Expired | Yellow = Expiring within 30 days

Customer	Vehicle	Type	Start Date	Length	Expiry Date	Days Remaining	Cost	Deductible	Items Covered	Action
Robert Garcia	2020 Ford Escape	Drive-Train	2025-09-20	7 months	2026-04-20	13	$900.00	$100.00	Engine, transmission, differential	Add Warranty
James Wilson	2018 Ford F-150	Drive-Train	2025-08-25	8 months	2026-04-25	18	$800.00	$100.00	Engine, transmission, differential, drive shaft	Add Warranty
James Wilson	2018 Ford F-150	Exterior	2025-08-25	9 months	2026-05-25	48	$400.00	$50.00	Paint, body panels, bumpers	Add Warranty
James Wilson	2020 Toyota Corolla	Electrical	2026-03-15	6 months	2026-09-15	161	$300.00	$50.00	Alternator, starter, wiring harness, battery	Add Warranty
Emily Brown	2021 Toyota RAV4	Exterior	2025-10-15	12 months	2026-10-15	191	$600.00	$50.00	Paint, body panels, bumpers, mirrors	Add Warranty
Kevin Nguyen	2019 Dodge Ram 1500	Interior	2025-10-25	12 months	2026-10-25	201	$400.00	$100.00	Seats, dashboard, carpet, headliner	Add Warranty
James Wilson	2020 Toyota Corolla	Drive-Train	2026-03-15	12 months	2027-03-15	342	$500.00	$75.00	Engine, transmission, differential	Add Warranty
Emily Brown	2021 Toyota RAV4	Drive-Train	2025-10-15	24 months	2027-10-15	556	$1,200.00	$75.00	Engine, transmission, differential, drive shaft, CV joints	Add Warranty
Kevin Nguyen	2019 Dodge Ram 1500	Drive-Train	2025-10-25	24 months	2027-10-25	566	$1,500.00	$50.00	Engine, transmission, differential, drive shaft, transfer case	Add Warranty
Kevin Nguyen	2019 Dodge Ram 1500	Exterior	2025-10-25	24 months	2027-10-25	566	$500.00	$75.00	Paint, body panels	Add Warranty

Late Payments (`late_payments.php`)

Shows customers with late payments, sorted worst first. Basically a credit risk report for the dealership.

JonesAuto Home Forms Reports Queries

Late Payment Customers

Customer	Phone	Total Payments	Late Payments	Avg Days Late	Action
Robert Garcia	403-555-1003	5	5	15.2	Record Payment
Ashley Taylor	403-555-1006	4	3	12.0	Record Payment
James Wilson	403-555-9999	6	2	8.5	Record Payment
Maria Santos	403-555-1004	4	1	3.0	Record Payment

4 Stage 4: Business Queries

The project needed at least 10 queries with at least 5 advanced ones. I ended up with 12 total: 5 simple and 7 advanced. They are all on a single queries page with a dropdown selector. You pick a query, hit submit, and the results show up in a table below.

For Query 3 there's also a text input so you can search by make. The SQL for each query is shown on the page along with the results, which helps with debugging and the demo.

4.1 Query 1: What cars do we have on the lot right now? (Simple)

This one pulls up every vehicle that is still available for sale using a basic SELECT with WHERE and ORDER BY. It's the quickest way for a salesperson to see what is on the lot without walking outside.

```
SELECT * FROM vehicles
WHERE status = 'available'
ORDER BY make, model
```

4.2 Query 2: Show me all our customers (Simple)

Just a quick way to look up any customer's contact info, using a basic SELECT with ORDER BY.

```
SELECT customer_id, first_name, last_name, phone, city, state
FROM customers
ORDER BY last_name
```

4.3 Query 3: Do we have any [make] cars in stock? (Simple)

This one takes a text input for the make, so the actual query uses the value the user types in and filters with a WHERE clause and AND condition. It's for when a customer calls and asks "do you have any Toyotas?" and you need a quick answer.

```
SELECT * FROM vehicles
WHERE make = '[make]' AND status = 'available'
```

4.4 Query 4: Who are our employees? (Simple)

Quick reference for who works here and what role they have. Just a basic SELECT with ORDER BY on two columns.

```
SELECT * FROM employees
ORDER BY role, last_name
```

4.5 Query 5: How many cars did we buy this month? (Simple)

Counts how many cars we bought this month and how much we spent total. Uses COUNT and SUM aggregate functions, and filters automatically based on the current date using MONTH/YEAR functions with CURDATE().

```
SELECT COUNT(*) as cars_bought, SUM(price_paid) as total_spent
FROM purchases
WHERE MONTH(purchase_date) = MONTH(CURDATE())
AND YEAR(purchase_date) = YEAR(CURDATE())
```

4.6 Query 6: Which cars did we buy below book price? (Advanced)

Useful for buyers to see which deals paid off. This uses a JOIN between vehicles and purchases to compare what we paid against the book value, and calculates the savings as a derived column.

```
SELECT v.make, v.model, v.year, v.book_price, p.price_paid,
       (v.book_price - p.price_paid) as savings
FROM vehicles v
JOIN purchases p ON v.vehicle_id = p.vehicle_id
WHERE p.price_paid < v.book_price
ORDER BY savings DESC
```

4.7 Query 7: What is our profit on each sale? (Advanced)

This was the hardest one to get right. Profit has to account for what we paid AND what we spent on repairs, so there's a correlated subquery that sums up repair costs for each car. COALESCE makes sure cars with no repairs don't break the calculation. The query uses multiple JOINs across sales, vehicles, and purchases.

```
SELECT v.make, v.model, v.year, s.sale_price, p.price_paid,
       (SELECT COALESCE(SUM(r.actual_cost),0) FROM repairs r
        WHERE r.purchase_id = p.purchase_id) as repair_costs,
       (s.sale_price - p.price_paid -
        (SELECT COALESCE(SUM(r.actual_cost),0) FROM repairs r
         WHERE r.purchase_id = p.purchase_id)) as profit
FROM sales s
JOIN vehicles v ON s.vehicle_id = v.vehicle_id
JOIN purchases p ON v.vehicle_id = p.vehicle_id
ORDER BY profit DESC
```

4.8 Query 8: Which customers have late payments? (Advanced)

Shows every late payment with the customer name, what car they bought, and how many days late they were. This uses a 4-table JOIN across payments, customers, sales, and vehicles, with DATEDIFF to calculate the days late and DISTINCT to avoid duplicates.

```
SELECT DISTINCT CONCAT(c.first_name, ' ', c.last_name) as customer_name,
       c.phone, v.make, v.model, v.year,
       DATEDIFF(pay.paid_date, pay.due_date) as days_late
FROM payments pay
JOIN customers c ON pay.customer_id = c.customer_id
JOIN sales s ON pay.sale_id = s.sale_id
JOIN vehicles v ON s.vehicle_id = v.vehicle_id
WHERE pay.paid_date > pay.due_date
ORDER BY days_late DESC
```

4.9 Query 9: Who is our top salesperson? (Advanced)

Groups sales by employee using GROUP BY to see who sold the most and brought in the most revenue, with COUNT and SUM aggregate functions. Top performer shows up first.

```
SELECT CONCAT(e.first_name, ' ', e.last_name) as salesperson,
       COUNT(s.sale_id) as num_sales,
       SUM(s.sale_price) as total_revenue,
       SUM(s.commission) as total_commission
FROM sales s
JOIN employees e ON s.employee_id = e.employee_id
GROUP BY e.employee_id
ORDER BY num_sales DESC
```

4.10 Query 10: Which warranties expire within 90 days? (Advanced)

This uses DATE_ADD to calculate the expiry date from start_date + length_months, then BETWEEN to check if it falls within the next 90 days. The DATEDIFF gives the exact number of days remaining so the dealership can follow up with customers before their coverage runs out. It's a 4-table JOIN across warranty_items, warranties, vehicles, and customers.

```
SELECT CONCAT(c.first_name, ' ', c.last_name) as customer_name,
       v.make, v.model, v.year,
       wi.warranty_type, wi.start_date,
       DATE_ADD(wi.start_date, INTERVAL wi.length_months MONTH)
           as expiry_date,
       DATEDIFF(DATE_ADD(wi.start_date,
           INTERVAL wi.length_months MONTH), CURDATE())
           as days_remaining
FROM warranty_items wi
JOIN warranties w ON wi.warranty_id = w.warranty_id
JOIN vehicles v ON w.vehicle_id = v.vehicle_id
JOIN customers c ON w.customer_id = c.customer_id
WHERE DATE_ADD(wi.start_date, INTERVAL wi.length_months MONTH)
      BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 90 DAY)
ORDER BY expiry_date
```

4.11 Query 11: Where did repair costs go over budget? (Advanced)

This groups repairs by vehicle using GROUP BY and uses HAVING to only keep the ones where the SUM of actual costs went over the SUM of estimates. The over_budget column is a calculated difference.

```
SELECT v.make, v.model, v.year,
       SUM(r.est_cost) as estimated_total,
       SUM(r.actual_cost) as actual_total,
       (SUM(r.actual_cost) - SUM(r.est_cost)) as over_budget
FROM repairs r
JOIN purchases p ON r.purchase_id = p.purchase_id
JOIN vehicles v ON p.vehicle_id = v.vehicle_id
GROUP BY p.purchase_id
HAVING SUM(r.actual_cost) > SUM(r.est_cost)
ORDER BY over_budget DESC
```

4.12 Query 12: Which customers bought more than one car? (Advanced)

Finds repeat buyers using GROUP BY with HAVING COUNT to filter for customers with more than one sale. Good for identifying loyal customers who might want to hear about new inventory.

```
SELECT CONCAT(c.first_name, ' ', c.last_name) as customer_name,
       c.phone,
       COUNT(s.sale_id) as cars_bought,
       SUM(s.sale_price) as total_spent
FROM customers c
JOIN sales s ON c.customer_id = s.customer_id
GROUP BY c.customer_id
HAVING COUNT(s.sale_id) > 1
ORDER BY cars_bought DESC
```

5 Assumptions and Design Decisions

Here are the main assumptions I made and the reasoning behind the bigger design decisions.

1. **Used `condition_desc` instead of `condition`**: MySQL has `condition` as a reserved word, so I couldn't use it as a column name without backticks everywhere. Renaming to `condition_desc` was the simplest fix. Not ideal but it works.

2. **Denormalized late payment stats on customers table**: The `num_late_payments` and `avg_days_late` columns on the customers table are technically redundant since they could be calculated from the payments table each time. But the payment form spec required showing these values, and recalculating on every page load with an aggregate query would be slower. I chose to store them directly and update them whenever a late payment is recorded.

Normalization trades some redundancy for query performance, and this is one of those trade-offs.

3. **`customer_id` duplicated on payments**: The payments table has `customer_id` even though you could get it by joining through sales. It's there so I can quickly look up all payments for a customer without an extra JOIN. Another deliberate denormalization.

4. **AUTO_INCREMENT surrogate keys everywhere**: Every table uses an INT AUTO_INCREMENT primary key instead of composite or natural keys. It's simpler for foreign key references and avoids issues if natural values change. I went with surrogate keys because they are straightforward.

5. **Tables created in FK dependency order**: The `db_setup.sql` file creates tables in the right order (employees and vehicles first, then purchases, etc.) so the foreign key constraints don't fail. This way the script runs straight through without needing deferred constraint checks.

6. **Alberta-specific sample data**: All the seed data uses Lethbridge, Calgary, and Medicine Hat locations with Alberta postal codes. Keeps it realistic for the course context.

7. **Deliberate late payment data in seed**: I included some payment records where `paid_date` is after `due_date` on purpose, so the late payment reports and queries actually have data to show during testing and the demo.

8. **Soft-delete with `is_active` flag**: Instead of actually deleting rows, all tables have an `is_active` column. When you "delete" a customer or employee, it just sets `is_active` to 0. This preserves referential integrity and keeps historical data intact. The trade-off is that every query in the system needs to filter on `is_active = 1`, which was a lot of extra work to add everywhere.

9. **Operations log for auditing**: The `operations_log` table records inserts, updates, and deletes with the old and new values. It's not part of any form or report, but it's there for debugging and accountability.

6 Challenges

There were a few things that gave me a lot of trouble during development.

1. **MySQL socket permissions on WSL**: When I first tried connecting to MySQL from PHP, it kept failing with a socket error. Turns out using "localhost" makes PHP try to connect through a Unix socket, and the permissions were wrong on WSL. Changing `config.php` to use `127.0.0.1` instead of `localhost` fixed it because that forces a TCP connection instead.

2. **Startup script failing on re-runs**: The first time you run the setup script everything works fine, but running it again would crash with "Table already exists" errors. I had to change the script to drop and recreate the entire database each time so it could be run repeatedly without issues.

3. **MySQL strict mode breaking queries**: MySQL's `ONLY_FULL_GROUP_BY` mode caused problems with the sale form's vehicle query. The query selected columns like make, model, and year, but not all of them were in the GROUP BY clause, so MySQL rejected it. I had to restructure the query to either include all selected columns in GROUP BY or use aggregate functions.
4. **Warranty form showing existing warranties**: I originally tried using JavaScript data attributes to store warranty info on the sale dropdown and display it when a sale was selected. That got really messy and buggy, so I switched to a page-reload approach where selecting a sale reloads the page and queries the database for existing warranties. Simpler and more reliable.
5. **Adding soft-delete everywhere**: Once I decided to use an `is_active` flag for soft-delete, I had to go through every single query in every file and add `WHERE is_active = 1` (or `AND is_active = 1` if there was already a WHERE clause). It was tedious and easy to miss one, which would cause deactivated records to show up in dropdowns or reports.

7 What I Learned

The biggest thing I took away from this project is how normalization actually works in practice. In class we walked through functional dependencies and decomposition on paper, but actually deciding where to denormalize for a real application was a different kind of challenge. It's one thing to identify a transitive dependency, it's another to decide whether the performance trade-off is worth it.

I had never used PHP before this course. The procedural mysqli pattern from the tutorials turned out to be pretty straightforward once I got the hang of it. The hardest part was debugging SQL errors since PHP just gives you a generic error string and you have to figure out which part of the query is wrong.

Drawing the ER diagram first and then mapping it to relational tables made the schema design way more structured than if I had just started creating tables. It forced me to think about relationships and cardinalities before writing any SQL.

Setting up the foreign keys caught several data integrity issues during testing. I tried inserting a sale for a `vehicle_id` that didn't exist and MySQL blocked it. That's exactly what referential integrity is supposed to do, and it's way better than having orphaned records.

Writing the advanced queries (especially Query 7 with the correlated subquery) pushed me to actually understand how JOINs and subqueries work together. The examples from class used the banking database, but applying those patterns to a different domain helped it click.

Implementing full CRUD operations (create, read, update, delete) on the customer and employee forms gave me a better understanding of how real applications work. The soft-delete pattern was interesting because it shows how you can "delete" data from the user's perspective while keeping it in the database for referential integrity. The downside is you have to remember to filter on `is_active` in every single query, which is easy to forget.

8 Future Improvements

If I had more time there are a few things I'd want to add. A login system would be the biggest one, since right now anyone can access everything. Buyers, salespeople, and managers should have different permissions. The reports could also use better sorting, like clicking column headers to re-sort. The forms use HTML tables for layout which doesn't look great on phones, so a responsive design would help. And some kind of inventory alert when the lot is getting full would be useful for the buyers at auctions.